

Node Pipeline Designer

Problem statement and objectives

Version 1.0 · Author: dhanoliya-ji

1. Context

Software that processes data in stages is usually described as a *pipeline*: a value enters, passes through a series of transformations, and leaves as a result. Data engineering tools (Airflow, Prefect, Dagster), machine-learning orchestration tools, ETL products and the newer LLM-chaining frameworks all share this shape. They also share a structural requirement that is rarely stated explicitly but is never optional:

A pipeline must be a directed acyclic graph.

Directed, because data flows one way — from a producer to a consumer. *Acyclic*, because a stage that ultimately depends on its own output can never begin: there is no order in which to run it. A cycle in a pipeline is not a slow pipeline or a wrong pipeline. It is a pipeline that cannot be scheduled at all.

The difficulty is that pipelines are increasingly **built visually**, by dragging boxes onto a canvas and drawing lines between them. A visual editor makes a cycle trivially easy to create — three connections in a triangle — and simultaneously makes it hard to see, because the loop may span parts of a large diagram the user is not looking at.

2. The problem

Building a pipeline visually surfaces four distinct problems, each of which this project addresses.

2.1 Structural correctness cannot be left to the user

A canvas will happily let someone draw $A \rightarrow B \rightarrow C \rightarrow A$. Nothing about the drawing looks wrong. The failure only appears when the pipeline is run, at which point the error message is typically a scheduler complaining about an unresolvable dependency — a message that names the symptom, not the mistake.

What is needed: the cycle must be detected before execution and reported in terms of the diagram the user drew, naming the nodes on the loop.

2.2 Structural correctness is not the same as being *wired* correctly

A pipeline can be a perfect DAG and still be nonsense. A Math node with only one of its two operands connected is acyclic. So is an HTTP node with no URL, a template referring to a variable nothing feeds, and a node sitting alone in the corner of the canvas connected to nothing.

These are not structural errors — a graph algorithm cannot see them, because they are facts about what each *kind* of node requires. They are, however, exactly the errors a user actually makes.

What is needed: a second, semantic layer of validation that understands what each node type needs, reported with the same precision as the structural layer.

2.3 Every new node type multiplies the work

The naive way to build a node editor is one component per node type. Nine node types then means nine components, nine sets of styles, nine places that read and write state, and nine places to update when the design changes. The cost of a tenth node type is the same as the cost of the first, which is the definition of a system that does not scale.

Worse, it makes *dynamic* nodes nearly impossible. A template node whose inputs are derived from the text typed into it — one socket per `{{variable}}` — needs its connection points to change as the user types, and to clean up the connections belonging to sockets that no longer exist.

What is needed: a node abstraction where a new node type is *data*, not code, and where a node's connection points may be a function of its own state.

2.4 A pipeline that cannot run is only half a design tool

Validation says a pipeline *could* work. It does not say what it *does*. The gap between the two is where most of the real mistakes live: a template that substitutes the wrong variable, a filter whose condition never matches, a branch that silently takes the path the author did not expect.

What is needed: an execution engine that runs the graph and reports what each node received, what it produced, and — for the branches not taken — why it did not run at all.

3. Objectives

#	Objective	How it is met	How it is verified
O1	Detect whether a pipeline is a DAG	Kahn's algorithm, <code>core/graph.py</code>	36 tests in <code>test_graph.py</code> ; chains, rings, diamonds, self-loops, disconnected components
O2	Name the nodes on a cycle, not just report that one exists	Iterative colour-marking DFS over the nodes the topological sort could not place	<code>TestCycleReporting</code> ; verified on multiple independent cycles
O3	Validate a pipeline semantically, per node type	A node-type registry with declared handles and required inputs, <code>core/validation.py</code>	23 tests in <code>test_validation.py</code> , one per rule
O4	Make adding a node type cost one object	Config-driven node abstraction, <code>nodeConfigs.js</code> rendered by a single <code>BaseNode</code>	Nine node types, zero bespoke components
O5	Support nodes whose sockets depend on their own state	<code>handles</code> may be a function of the node's data; orphaned edges are pruned when sockets disappear	<code>textVariables.test.js</code> , <code>store.test.js</code> handle-pruning suite
O6	Execute a pipeline and report every step	Single-pass topological engine, <code>core/engine.py</code>	58 tests in <code>test_engine.py</code>
O7	Handle conditional branches correctly	A handler may omit an output; the engine treats that as "this path carried nothing" and skips the subtree	<code>TestBranching</code> , including skip propagation

#	Objective	How it is met	How it is verified
O8	Evaluate user-supplied expressions without handing over the interpreter	AST walker over a whitelisted subset, <code>core/expressions.py</code>	42 tests, including a suite of attempted escapes
O9	Prove the graph layer scales linearly	Benchmarks across four graph shapes and two orders of magnitude	<code>notebooks/analysis.ipynb</code> ; fitted exponents 0.93–1.01
O10	Make the tool usable for real work	Undo/redo, save/load, JSON import/export, auto-layout, keyboard shortcuts, live cycle prevention	94 frontend tests

Non-objectives

Stated explicitly, because a scope that is not bounded is not a scope.

- **Not a production orchestrator.** There is no scheduling, retry policy, distributed execution or persistence beyond a JSON file. The contribution is the editor and its correctness layer.
- **Not an LLM product.** The LLM node returns a clearly-labelled simulated response. Wiring a real provider is a change to one function body; holding someone's API key is not what this project is about.
- **Not multi-user.** No accounts, no collaborative editing, no server-side storage of pipelines.

4. Requirements

4.1 Functional

ID	Requirement
F1	Nodes can be dragged from a palette onto a canvas and connected by dragging between sockets
F2	The system reports whether the pipeline is a DAG, and names the nodes of any cycle
F3	The system reports semantic problems (missing required inputs, unknown sockets, ambiguous fan-in, isolated nodes, unconfigured nodes) with a severity and the element responsible
F4	A connection that would create a cycle is refused as it is drawn
F5	The pipeline can be executed, producing a per-node record of status, output and duration
F6	Conditional nodes route to exactly one branch; the untaken branch's subtree is skipped, not failed
F7	A pipeline can be saved to the browser, exported to JSON and imported from JSON
F8	Edits can be undone and redone

ID	Requirement
F9	The canvas can be arranged automatically in dependency order
F10	A new node type can be added by adding one configuration object and one handler

4.2 Non-functional

ID	Requirement	Target	Measured
N1	Validation is linear in the size of the graph	exponent ≈ 1.0	0.93–1.01 across four shapes
N2	A large pipeline validates interactively	< 50 ms at 2,000 nodes	~10 ms
N3	Cycle detection does not recurse	no stack limit	iterative DFS, tested at 5,000 nodes
N4	User expressions cannot reach the host	no filesystem, network or interpreter access	10 escape attempts, all refused
N5	The backend survives a bad request	no unhandled exception	handler failures are caught per node
N6	The code is covered by tests	> 85% statements	92% backend, 275 tests overall

5. Approach

5.1 Two layers of validation, deliberately separate

Structure and semantics are answered by different passes because they are different questions with different inputs.

The structural pass (`analyse`) knows only about ids and edges. It runs Kahn's algorithm: repeatedly remove a node with no remaining incoming edges and decrement its successors. If the queue empties while nodes remain, those nodes are on or downstream of a cycle — a graph is acyclic exactly when the topological sort covers it. The same pass records each node's *level* (one past its deepest predecessor), which doubles as the column index the auto-layout uses.

When the sort does not cover the graph, a second, iterative DFS runs **only over the nodes it could not place**, marking nodes on the current path; when the walk meets a node already on the path, the slice of the path from that node is a cycle. It is iterative rather than recursive so a long chain cannot exhaust the interpreter's stack.

The semantic pass (`validate`) knows what each node type is. A registry declares, per type, its input and output sockets and which inputs are mandatory — and a type may compute its sockets from a node's data, which is what lets a Text node derive one input per `{{variable}}`. Each rule produces an `Issue` carrying a stable code, a severity and the id of the node or edge responsible, so the interface can highlight the exact element rather than display a paragraph.

5.2 One node abstraction, configured by data

A node type is a plain object: a label, an icon, an accent colour, a list of field specifications, and a list of sockets. A single `BaseNode` component renders any of them. Because `fields` and `handles` may each be a *function* of the node's live state, a node's shape can follow its own content — and when a socket disappears, the store prunes the edges attached to it so nothing dangles.

The result is that adding a node type touches two files and writes no JSX, no CSS and no state-management code.

5.3 Execution as a single topological pass

Because the nodes are already in topological order, every predecessor of a node has run by the time the node does. One pass is therefore sufficient: no scheduler, no re-entry, no fixed-point iteration.

Branching falls out of the same rule rather than being bolted on. A handler returns a value per output socket and **may omit one**; an omitted socket means nothing flowed that way. A node whose every incoming edge came up empty is *skipped* rather than run with missing data, and because skips are recorded as the pass proceeds, they propagate down the whole subtree automatically.

5.4 Expressions without an interpreter

The Condition node lets a user type an expression. Passing that to `eval` would hand whoever opens the app the entire Python runtime. Instead the expression is parsed to an AST and walked by hand, executing only a whitelisted set of node types. There is no attribute access, no subscripting, no imports, and only a fixed list of pure functions may be called — so there is no path from an expression to the filesystem, the network or the process.

6. Evaluation

The project is evaluated on evidence rather than assertion.

Question	Evidence
Is the DAG logic correct?	181 backend tests covering 92% of statements, including adversarial graph shapes
Does it scale as claimed?	Fitted power-law exponents of 0.93–1.01 over 10→2,000 nodes, four graph shapes (<code>notebooks/analysis.ipynb</code>)
Does validation catch real mistakes?	A deliberately broken pipeline produces 13 issues across 9 distinct rules
Is the expression evaluator safe?	10 escape attempts — <code>__import__</code> , <code>open</code> , attribute access, <code>exec</code> , <code>globals</code> — all refused
Does the frontend logic hold up?	94 tests across graph utilities, layout, serialisation and store behaviour

A finding worth recording

Benchmarking exposed a defect the test suite could not: validating a 2,000-node *ring* took 38 ms against 9 ms for a random DAG of the same size. Each node on a cycle received an issue whose message named the entire cycle, so a v -node ring built v strings of v ids — $O(v^2)$ inside an otherwise linear pass. Every unit test used a small graph, so the quadratic was invisible.

The message is now constructed once per cycle and abbreviated past eight nodes, returning the same case to 9.6 ms. This is the argument for measuring rather than assuming, and the reason the analysis notebook is part of the deliverable rather than an afterthought.

7. Deliverables

Deliverable	Location
Visual editor	<code>frontend/</code> — React, React Flow, Zustand
Validation and execution API	<code>backend/</code> — FastAPI, Pydantic
Test suites	<code>backend/tests/</code> (181), <code>frontend/src/**/*.test.js</code> (94)
Performance analysis	<code>notebooks/analysis.ipynb</code> , <code>backend/benchmarks/</code>
Architecture notes	<code>docs/ARCHITECTURE.md</code>
API reference	<code>docs/API.md</code>
This document	<code>docs/PROBLEM_STATEMENT.md</code> and <code>.pdf</code>

8. Glossary

DAG — Directed Acyclic Graph. A graph whose edges have direction and which contains no path from any node back to itself.

Topological sort — An ordering of a DAG's nodes such that every node appears before all nodes it points to. A graph has one if and only if it is acyclic.

Kahn's algorithm — A topological sort that repeatedly removes nodes with no remaining incoming edges. Runs in $O(V + E)$.

Handle / socket — A named connection point on a node. Edges join a source handle on one node to a target handle on another.

Node type registry — The mapping from a node type's name to its declared sockets, required inputs, and execution handler.